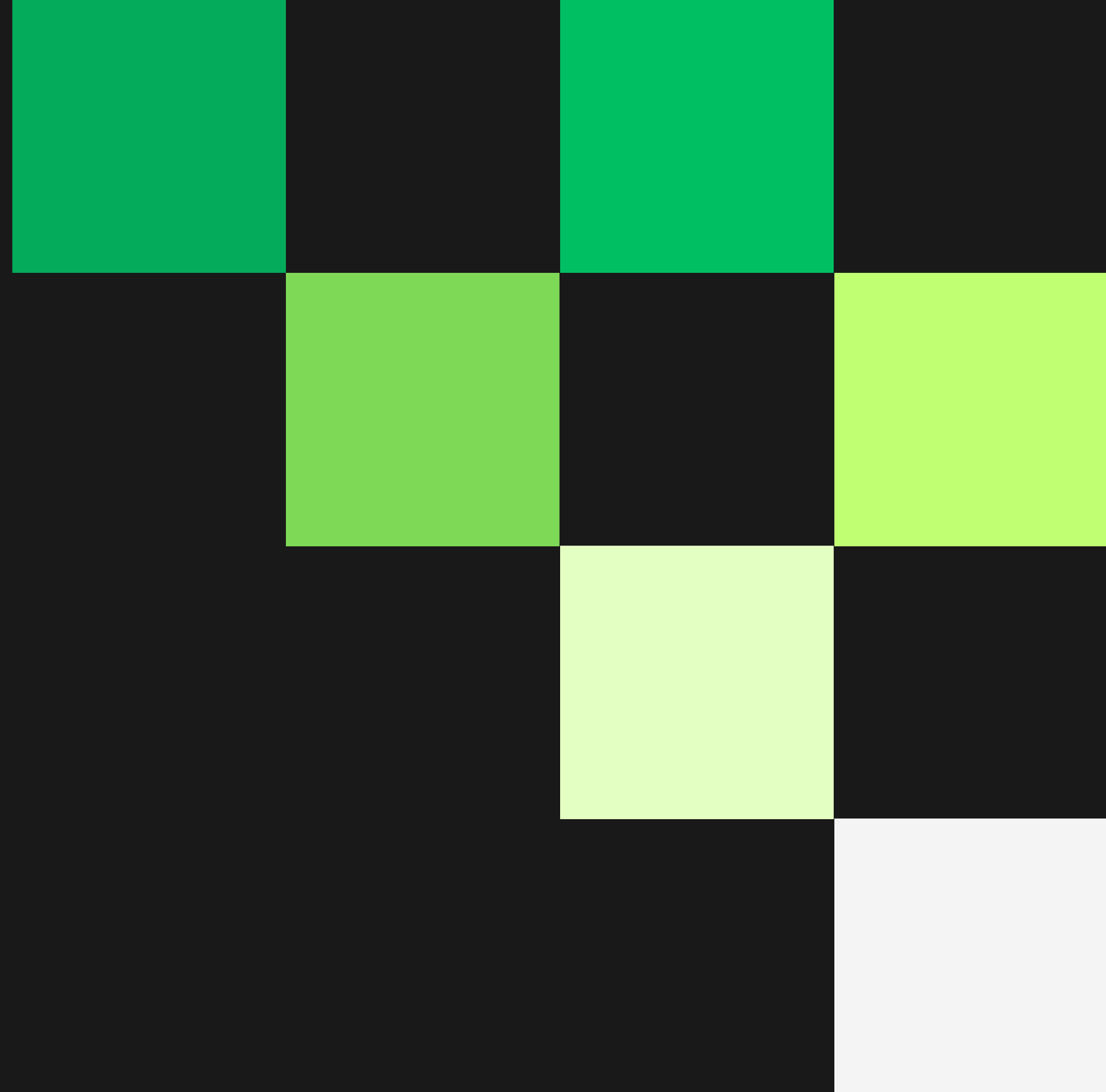


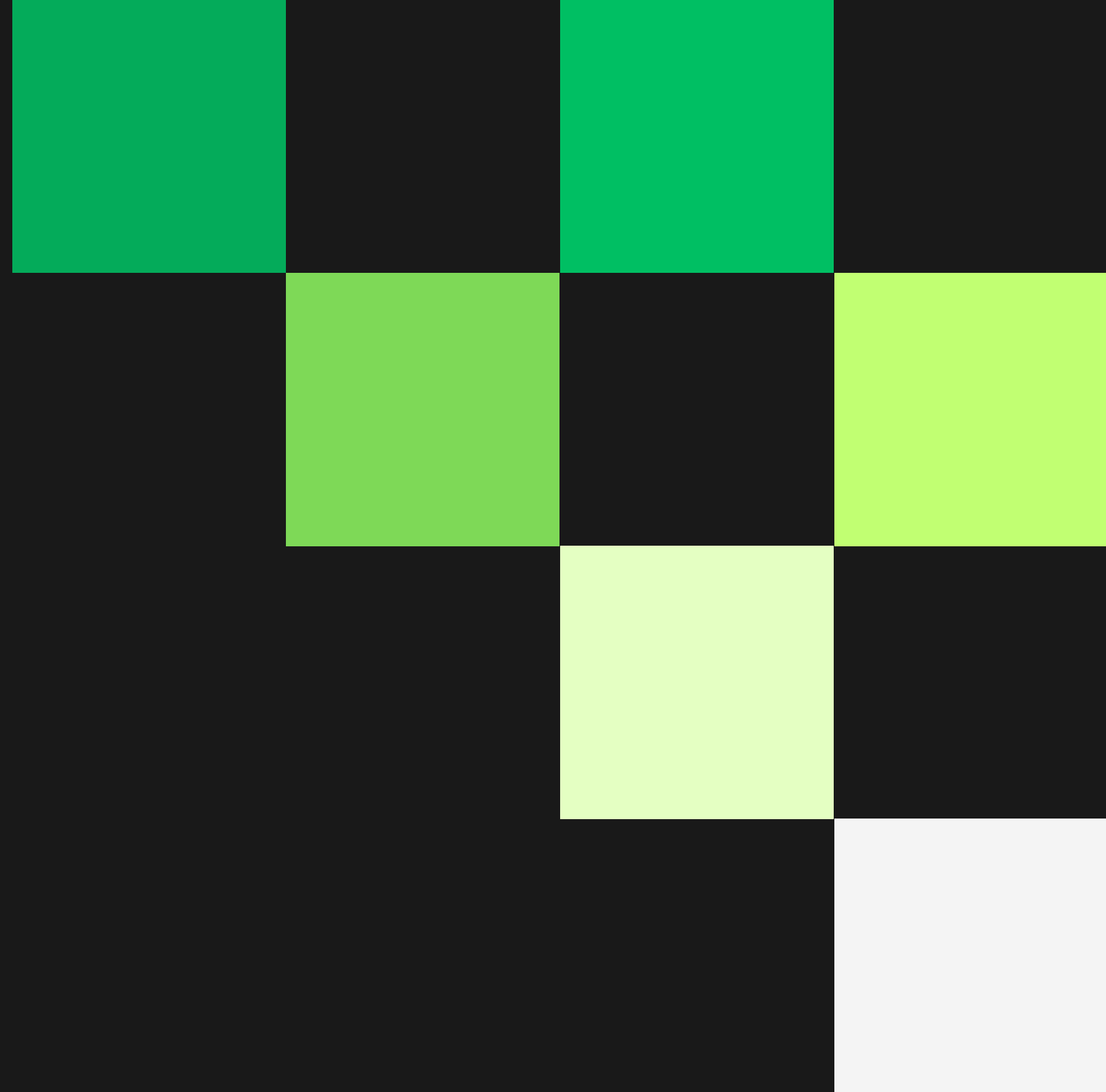
Felipe Colombo
Curso de Informática – SATC

Back End



Felipe Colombo
Curso de Informática – SATC

Back End 2



Funcionamento do PHP

O PHP é uma linguagem que atua do lado do servidor, recebendo requisições e gerando um HTML novo com base nos dados que foram solicitados. Ele funciona na seguinte sequência:

1. O usuário faz uma requisição.
2. O PHP no servidor recebe a requisição, lê o que foi pedido e realiza as ações necessárias (se conectar com um banco, fazer um comando sql, etc).
3. O PHP gera um novo conteúdo HTML.
4. O servidor pega o conteúdo HTML e o retorna para o usuário, mostrando o novo HTML no navegador.

PHP + Js

Utilizando JavaScript em conjunto com PHP, é possível manter as funcionalidades de Front-end e Back-end separadas, garantindo uma melhor organização dos projetos e fazendo com que o PHP não precise gerar sempre um novo HTML para as páginas.

1. O usuário interage com algo da página;
2. O JavaScript envia uma requisição para o servidor pedindo apenas alguns dados;
3. O PHP no servidor busca esses dados e os envia de volta, sem precisar enviar um novo HTML inteiro;
4. O JavaScript recebe esses dados e os atualiza na página HTML, sem precisar recarregar a página.

Iniciando

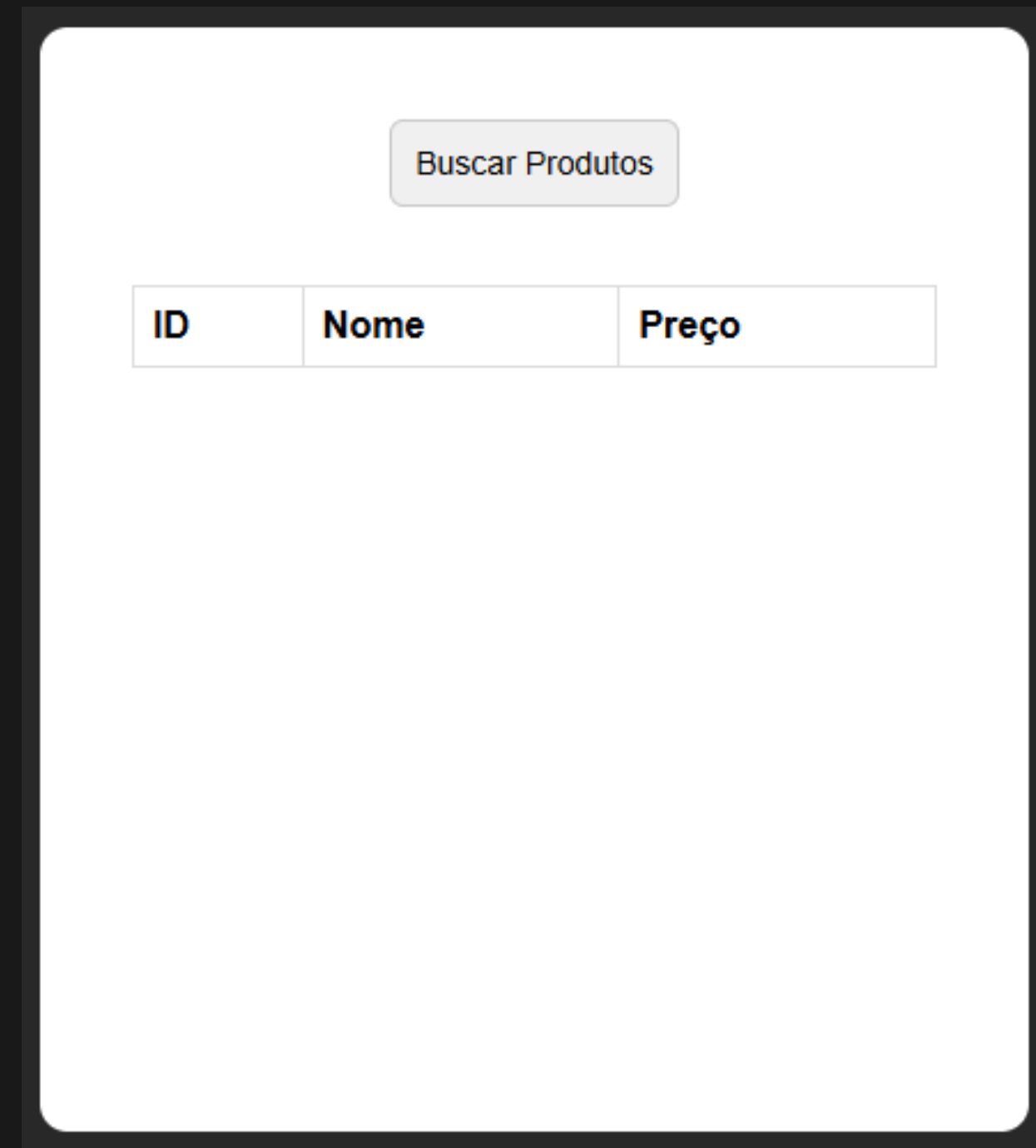
Estarei passando um mini projeto que contém:

- 1 arquivo de conexão com banco
- 1 arquivo com uma função de trazer todos os registros
- 1 arquivo com a parte visual
- 1 arquivo com o SQL de criação do banco de dados

Esse é um projeto puramente de PHP e HTML. Quando eu clico no botão de buscar, o PHP busca os dados do banco e cria uma nova página HTML, com os dados atualizados.

Para um projeto simples, isso é algo ok. Mas para projetos maiores, usar apenas o PHP pode trazer algumas desvantagens:

- Ter que recarregar toda uma página com muita coisa pode ser demorado;
- Você pode perder dados que já estão na página após ela recarregar;
- Menos seguro pois o PHP está à vista, junto do código HTML.



ID	Nome	Preço
----	------	-------

Js + PHP

Para adicionarmos Js no nosso projeto, precisamos fazer algumas mudanças.

No **menu.php**:

```
<body>
  <div class="container">
    <button id="buscar">Buscar Produtos</button>
    <br>

    <table id="resultado">
      <tr>
        <th>ID</th>
        <th>Nome</th>
        <th>Preço</th>
      </tr>
    </table>
  </div>
</body>
<script src="js\script.js"></script>
```

- Vamos **apagar todo o código PHP** desse arquivo. Podemos até renomear ele para .html
- No lugar do form, vamos colocar um **botão com id buscar**.
- Vamos colocar o id **resultado** na nossa table
- Vamos chamar nosso **script.js depois do body**, para ele rodar apenas quando o body já estiver carregado.

Js + PHP

Vamos estar criando um novo arquivo, que tem a função de ser uma “ponte” entre a nossa função de listar os dados do PHP, e o nosso script JS que criaremos mais pra frente.

Crie uma nova pasta chamada **api**, e dentro dela, um arquivo chamado **produtos.php**

```
<?php
require_once __DIR__ . "../model/produto.php";

$produtos = Produto::listar();

header("Content-Type: application/json");

echo json_encode($produtos);

?>
```

- Esse código vai ser responsável só por rodar a função de listar, e retornar um texto em formato **json** com os dados.
- Json é um formato usado para colocar dados organizados em formato de texto. Exemplo:

```
"alunos": [
  {
    "nome": "guilherme",
    "idade": 18,
    "curso": "informática"
  },
  {
    "nome": "ana",
    "idade": 17,
    "curso": "química"
  }
]
```

Js + PHP

Agora para a parte importante – nosso código JavaScript que vai pegar os dados do PHP e colocar eles na tela.

```
var botao = document.getElementById("buscar");

botao.onclick = function(){
    fetch("../api/produtos.php")
    .then(function(resposta){
        return resposta.json();
    })
}
```

- Vamos primeiro pegar o nosso elemento de botão com o id “buscar”, e adicionar uma função no clique dele com o onclick.
- Vamos usar a função **fetch** para chamar aquele código que acabamos de fazer. Essa função serve para fazer **requisições** e trazer dados do PHP.
- Depois do fetch, usamos o **then** para fazermos algo com os dados que o fetch trouxe. Nesse caso, vamos pegar apenas o **json** que foi retornado.

Js + PHP

Depois, usaremos mais um `.then` para pegar esses dados do Json, e criar na tela linhas de tabela usando esses dados.

```
    })
    .then(function(produtos){

        var html = "";

        produtos.forEach(function(produto){

            html += "<tr>";
            html += "<td>"+produto.id+"</td>";
            html += "<td>"+produto.nome+"</td>";
            html += "<td>R$ "+produto.preco+"</td>";
            html += "</tr>";

        });

        document.getElementById("resultado").innerHTML = document.getElementById("resultado").innerHTML + html;

    });

}
```

Js + PHP

Vimos como o PHP manda dados para o JavaScript.
Agora, vamos checar como podemos mandar dados do JavaScript para o PHP.
Então, faremos uma nova funcionalidade – Inserir.

No seu model > Produto.php, crie a seguinte função:

```
public static function inserir($nome, $preco){  
    global $conexao;  
  
    $sql = "INSERT INTO produtos(nome,preco) VALUES('$nome','$preco')";  
  
    return $conexao->query($sql);  
}
```

Js + PHP

Agora, precisamos criar o código que liga o PHP com o JS.
Dentro da pasta API, crie um novo código php - **inserir.php**

```
<?php

require_once __DIR__ . '/../model/produto.php';

$nome = $_POST["nome"];
$preco = $_POST["preco"];

// aqui dá pra fazer algumas validações, checar se algo
// está vazio, etc

if(Produto::inserir($nome,$preco)){

    echo "ok";

}else{

    echo "erro";

}

?>
```

Js + PHP

No código html, precisamos colocar os campos que iremos usar para adicionar dados na tabela: nome e preço.

Para isso, acima do botão “buscar produtos”, vamos adicionar o seguinte código:

```
<h2>Cadastrar produto</h2>

<input type="text" id="nome" placeholder="Nome">

<input type="number" id="preco" placeholder="Preço">

<button id="salvar">Cadastrar</button>

<br>
```

Js + PHP

O próximo passo será no JS. Porém, antes de criarmos a função de inserir, vamos dar uma ajeitada nele:

- Pegue todo o código que estava no **function** do `botao.onclick`, e coloque em uma função chamada **listarProdutos**
- Dentro do function do `botao.onclick`, agora chame o **listarProdutos**

```
// declaração de variaveis/elementos
var botao = document.getElementById("buscar");

// funções
// aqui dentro vai o que estava antes no
// function(){ do botao
> function listarProdutos(){ ...
}

// uso das funções
botao.onclick = function(){
|   listarProdutos();
}
```

Por que fazer isso?

É uma boa prática de código – dividir o script em “partes” que fazem coisas diferentes.

- No início do script temos nossas declarações de variáveis;
- No meio temos nossas declarações de funções;
- No fim temos o uso dessas funções

Js + PHP

O próximo passo será no JS. Porém, antes de criarmos a função de inserir, vamos dar uma ajeitada nele:

- Pegue todo o código que estava no **function** do botao.onclick, e coloque em uma função chamada **listarProdutos**
- Dentro do function do botao.onclick, agora chame o **listarProdutos**

```
function inserirProduto(){
    var nome = document.getElementById("nome").value;
    var preco = document.getElementById("preco").value;

    fetch("../api/inserir.php",{
        method:"POST",
        headers:{
            "Content-Type":"application/x-www-form-urlencoded"
        },
        body:"nome="+nome+"&preco="+preco
    })
}
```

Js + PHP

O próximo passo será no JS. Porém, antes de criarmos a função de inserir, vamos dar uma ajeitada nele:

- Pegue todo o código que estava no **function** do botao.onclick, e coloque em uma função chamada **listarProdutos**
- Dentro do function do botao.onclick, agora chame o **listarProdutos**

```
.then(function(resposta){
    return resposta.text();
})

.then(function(retorno){
    if(retorno=="ok"){
        alert("Produto cadastrado.");
        document.getElementById("nome").value="";
        document.getElementById("preco").value="";
    }
    else{
        alert("Erro ao cadastrar.");
    }
});
}
```

Js + PHP

O próximo passo será no JS. Porém, antes de criarmos a função de inserir, vamos dar uma ajeitada nele:

- Pegue todo o código que estava no **function** do botao.onclick, e coloque em uma função chamada **listarProdutos**
- Dentro do function do botao.onclick, agora chame o **listarProdutos**

```
// declaração de variaveis/elementos
var botaoBuscar = document.getElementById("buscar");
var botaoInserir = document.getElementById("salvar");
```

```
// uso das funções
botaoBuscar.onclick = function(){
    listarProdutos();
}

botaoInserir.onclick = function(){
    inserirProduto();
}
```

Trabalho 1

Em **trio** – usando como base o que vimos nas aulas de PHP + JS, já temos como fazer **buscas de dados** e **envio de dados** do JS para o PHP. Com isso:

- Crie uma nova tabela no banco de dados (o tema fica a sua escolha);
- Crie um CRUD completo para essa tabela (busca, cadastro, atualizar, excluir), com o back feito em PHP;
 - Dá para usar o **inserir** que fizemos em sala como base para **atualizar** e **excluir**.
- Crie páginas HTML, ligando-as ao PHP via JavaScript;
 - As páginas podem ser mais simples, mas usem algo diferente da página passada em aula;

Critérios de avaliação:

- Back-end (maior peso), front-end (menor peso)
- Comportamento em sala;
- Identificação no uso de IA vai gerar penalidades